SOFTWARE REQUIREMENTS SPECIFICATION

OnTime — Real-Time Bus Tracking & Booking System

Document Version: 1.0.0
Date: March 2026
Status: Production Release
Document Type: Software Requirements Specification (SRS)
Standard: IEEE 830 / ISO/IEC 25010

CONFIDENTIAL — For Internal Use Only

Table of Contents

Table of Contents 2
1. Introduction 4
  1.1 Purpose 4
  1.2 Project Scope 4
  1.3 Definitions and Acronyms 4
  1.4 Document Overview 5
2. Overall Description 6
  2.1 Product Perspective 6
  2.2 User Classes and Characteristics 6
  2.3 Operating Environment 6
  2.4 Design and Implementation Constraints 6
  2.5 Assumptions and Dependencies 6
3. System Architecture 8
  3.1 High-Level Architecture 8
  3.2 Component Breakdown 8
  3.3 Data Flow: Real-Time GPS Tracking 8
  3.4 Data Flow: Delay Notification 9
  3.5 Data Flow: Ticket Booking 9
4. Functional Requirements 10
  4.1 Authentication & Authorization 10
  4.2 Live Bus Tracking 10
  4.3 Trip Management (Driver) 11
  4.4 Ticket Booking (Passenger) 11
  4.5 Route & Stage Management (Admin) 11
  4.6 Fleet & Fare Management (Admin) 12
5. Non-Functional Requirements 13
  5.1 Performance 13
  5.2 Security 13
  5.3 Reliability & Availability 13
  5.4 Usability 13
  5.5 Maintainability 14
6. Data Model 15
  6.1 Entity Relationship Overview 15
  6.2 Collections 15
    User 15
    Bus 15
    Route 15

Stage 16
Trip 16
Booking 16
BusTypeFare 17
7. API Reference 18
7.1 Auth — /api/auth 18
7.2 Routes — /api/routes 18
7.3 Buses — /api/buses 18
7.4 Trips — /api/trips 18
7.5 Bookings — /api/bookings 19
7.6 Stages — /api/stages 19
7.7 Fare — /api/fare 19
7.8 Bus Type Fares — /api/bus-types 19
8. Socket.IO Event Catalog 21
8.1 Connection Lifecycle 21
8.2 Client → Server Events 21
8.3 Server → Client Events (Broadcast) 21
9. Deployment & Infrastructure 22
9.1 Frontend — Vercel 22
9.2 Backend — Railway 22
9.3 Database — MongoDB Atlas 22
9.4 Environment Variables Reference 22
Client (client/.env) 22
Server (server/.env) 22
10. Known Limitations & Future Roadmap 24
10.1 Known Limitations (v1.0) 24
10.2 Future Roadmap 24
Appendix A: Fare Calculation Formula 26
A.1 Stage-to-Stage Fare 26
A.2 Default Fare Rates 26
Appendix B: ETA Calculation 26
B.1 Haversine Formula 26
Appendix C: Seat Numbering Layout 26

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the complete functional, non-functional, and system requirements for OnTime — a full-stack real-time bus tracking and ticket booking platform. This document serves as the authoritative reference for developers, testers, project managers, and stakeholders throughout the software development lifecycle.

1.2 Project Scope

OnTime addresses the core problems faced by public bus commuters in Karnataka, India:

- Inability to know real-time bus location before arriving at a stop
- Manual, paper-based ticketing with no seat assurance
- Lack of proactive delay notification for waiting passengers
- No unified dashboard for fleet operators to monitor buses

The system delivers:

- Live GPS bus tracking on an interactive map with ETA calculation
- Stage-to-stage dynamic fare computation and multi-seat e-ticket booking
- Real-time delay alerts pushed to all connected passengers instantly
- Separate role-based interfaces for passengers, drivers, and administrators
- Admin tools for route creation (via Mapbox), stage pinning (via Leaflet), and fleet management

1.3 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification
JWT	JSON Web Token — stateless authentication mechanism
GPS	Global Positioning System — real-time location data from browser geolocation API
ETA	Estimated Time of Arrival — calculated from Haversine distance and current speed
Stage	A named bus stop pinned on the route map with a distance from origin
Route	A defined path between two cities with source/destination coordinates and a stored GeoJSON polyline
Trip	A single operational run of a bus on a route — has a start time, end time, and real-time location history
Booking	A passenger reservation for one or more seats on a specific bus with a unique ticket ID and QR code
Socket.IO	WebSocket-based real-time bidirectional event library used for GPS updates and delay notifications
OSRM	Open Source Routing Machine — used to calculate driving distance between GPS coordinates
DXA	Document exchange units — 1440 DXA = 1 inch (DOCX measurement unit)

1.4 Document Overview

This SRS is organized as follows:

1. Introduction — purpose, scope, definitions
2. Overall Description — product perspective, user roles, constraints
3. System Architecture — component diagram and technology stack
4. Functional Requirements — detailed per module
5. Non-Functional Requirements — performance, security, usability
6. Data Model — entity descriptions and relationships
7. API Reference — all endpoints with auth levels
8. Socket.IO Event Catalog — real-time event contracts
9. Deployment & Infrastructure — environment and hosting
10. Known Limitations & Future Roadmap

2. Overall Description

2.1 Product Perspective

OnTime is a standalone web application with a React SPA frontend communicating with a Node.js/Express REST API backend. Real-time GPS updates and delay notifications are delivered via a persistent Socket.IO WebSocket layer. Persistent data is stored in MongoDB Atlas. The system integrates with three external APIs:

- Mapbox — route geocoding, polyline routing, and interactive admin map
- OSRM — road-network distance calculation for stage-to-origin measurements
- OpenStreetMap / Nominatim — stage management map search

2.2 User Classes and Characteristics

Role	Creation Method	Primary Interface	Key Capabilities
Passenger	Self-register (public)	Booking, Tracking, Dashboard	Search routes, book tickets, track buses, receive delays
Driver	Self-register (public)	Driver Dashboard	Start/end trips, share GPS, report delays
Admin	Database / seed only	Admin Dashboard, Route/Stage/Bus/Fare mgmt	Full fleet and system management

> *Security Note: Admin accounts cannot be created via the registration endpoint. Any attempt to register with role=admin is rejected at the server with HTTP 403, regardless of what the client sends.*

2.3 Operating Environment

- Browser: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+ (WebSocket + Geolocation API required)
- Mobile: iOS Safari 14+, Android Chrome 90+ (responsive layout down to 320px)
- Node.js: v18+ (ES Modules, native fetch)
- MongoDB Atlas: M0 (free tier) or above

2.4 Design and Implementation Constraints

- All frontend code uses ES Modules (type: "module" in package.json)
- Tailwind CSS JIT requires a safelist for all dynamically-composed class strings
- GPS location sharing requires HTTPS in production (browser security requirement)
- Mapbox token must be kept server-side or rate-limited to prevent abuse in production
- ETA calculation is Haversine (straight-line) — not road-network — giving indicative rather than precise estimates

2.5 Assumptions and Dependencies

- Bus drivers carry a device with a modern browser and stable internet connection

- MongoDB Atlas cluster is in the same region as the Railway hosting for low latency
- OSRM public instance (router.project-osrm.org) is available; in production, a private instance should be deployed
- Mapbox free tier is sufficient for admin map usage in early deployment

3. System Architecture

3.1 High-Level Architecture

OnTime follows a three-tier architecture: a React SPA (Presentation), an Express REST + Socket.IO API (Business Logic), and MongoDB Atlas (Data). The frontend and backend are deployed independently and communicate via HTTP/WebSocket.

> *Architecture Pattern: Monolithic API with real-time layer. The Express server handles both REST and Socket.IO on the same HTTP server instance. Socket.IO state is managed in-memory (single server instance). For horizontal scaling, a Redis adapter would be required.*

3.2 Component Breakdown

Layer	Component	Technology	Responsibility
Presentation	SPA Shell	React 19 + Vite 7	Routing, auth state, layout, global toasts
Presentation	Auth Pages	React + Zustand	Login, Register with role guard
Presentation	TrackBus Map	Leaflet + React-Leaflet	Live bus markers, route polyline, ETA panel, mobile drawer
Presentation	Booking Wizard	React multi-step	Route → Bus → Stage → Seat → Confirm flow
Presentation	Admin Maps	Mapbox GL JS	Route creation with geocoding and polyline
Presentation	Stage Editor	Leaflet + Nominatim	Click-to-pin bus stops with OSRM distance
Presentation	E-Ticket	React + qrcode	QR ticket generation, native print
Presentation	Notifications	Socket.IO client + Notification API	Bell dropdown, browser push, delay banner
API	REST Router	Express 4	8 route modules, JWT/role middleware
API	Socket Handler	Socket.IO Server	GPS broadcast, trip events, delay broadcast
API	Auth Middleware	jsonwebtoken + bcryptjs	JWT verify, role enforcement
API	Rate Limiter	express-rate-limit	Global 200/15min, auth 20/15min, fare 30/1min
Data	MongoDB Atlas	Mongoose 8	Schemas, indexes, populated queries
External	Mapbox API	REST	Geocoding, Directions with polyline + alternatives
External	OSRM	REST	Road-network distance for stage creation
External	Nominatim	REST	Place search for stage map zoom

3.3 Data Flow: Real-Time GPS Tracking

11. Driver opens Dashboard → browser prompts for geolocation permission
12. Driver clicks "Start Trip" → POST /api/trips/start → Bus.isOnTrip = true
13. navigator.geolocation.watchPosition fires every ~3–5 seconds
14. Client emits driver:location-update {busId, lat, lng, speed} via Socket.IO
15. Server socket handler updates Bus.currentLocation in MongoDB
16. Server broadcasts bus:location-updated to all connected clients

17. TrackBus page receives event → updates bus marker coordinates in React state
18. ETA recalculated client-side using updated coordinates and speed

3.4 Data Flow: Delay Notification

19. Driver submits delay form → POST /api/trips/:id/delay {delayMinutes, delayReason}
20. Server verifies driver owns the trip (trip.driverId === req.user._id)
21. Server persists delay to Trip document in MongoDB
22. Server calls io.emit("trip:delay", payload) — broadcast to ALL connected clients
23. TrackBus: if selected bus matches busId → DelayBanner renders with fixed positioning
24. useNotifications hook: adds notification to bell dropdown state
25. Browser Notification API fires if permission granted and tab not focused

3.5 Data Flow: Ticket Booking

26. Passenger selects route → GET /api/buses?routeId=&status=active
27. Passenger selects bus → GET /api/stages/:routeId
28. Passenger selects boarding and destination stages
29. GET /api/fare/calculate?fromStageId=&toStageId=&busId= → returns fare (₹)
30. Fare validated (> 0) before allowing seat selection
31. GET /api/bookings/seats/:tripId → load already-booked seats from active trip
32. Passenger selects seats from 47-seat grid (booked seats disabled)
33. POST /api/bookings → booking created, ticketId generated, Trip.bookedSeats updated
34. E-Ticket component renders with QR code → passenger downloads/prints

4. Functional Requirements

Priority: P1 = Must Have | P2 = Should Have | P3 = Nice to Have

4.1 Authentication & Authorization

ID	Requirement	Priority	Notes
FR-AUTH-01	Public registration for passenger and driver roles only	P1	Admin blocked at API (HTTP 403) and UI (removed from dropdown)
FR-AUTH-02	JWT issued on login/register, 7-day expiry, stored in localStorage	P1	Token sent as Bearer in Authorization header
FR-AUTH-03	Protected routes redirect unauthenticated users to /login	P1	Via ProtectedRoute component
FR-AUTH-04	Role-based guards redirect wrong roles to their correct dashboard	P1	allowedRoles prop on ProtectedRoute
FR-AUTH-05	Token expiry triggers auto-logout and redirect to /login	P1	Axios 401 interceptor in api.js
FR-AUTH-06	Password change requires current password and min 6 chars	P1	PUT /auth/profile/password; logs out on success
FR-AUTH-07	User can update name and phone from Profile page	P2	Email is immutable

4.2 Live Bus Tracking

ID	Requirement	Priority	Notes
FR-TRACK-01	Show all active buses on Leaflet map with custom bus icon	P1	GET /buses?status=active on mount
FR-TRACK-02	Bus markers update position in real-time via Socket.IO	P1	bus:location-updated event
FR-TRACK-03	Selecting a bus flies map to its coordinates (zoom 16)	P1	flyTo with 1.5s duration
FR-TRACK-04	Route polyline drawn on map when bus selected	P1	Mapbox GeoJSON preferred, stops[] fallback
FR-TRACK-05	ETA calculated using Haversine distance and GPS speed (m/s → km/h)	P2	etaCalculator.js utility
FR-TRACK-06	Buses with no active trip show clear "no live location" indicator	P2	Both in sidebar list and detail overlay
FR-TRACK-07	Mobile sidebar collapses to hamburger drawer overlay	P1	Fixed w-80 sidebar hidden on <md
FR-TRACK-08	Socket disconnect triggers persistent toast; reconnect dismisses it	P2	id: track-disconnect, duration: 0
FR-TRACK-09	Delay banner shown above map when active bus reports delay	P1	Fixed position, z-[9998] above Leaflet

4.3 Trip Management (Driver)

ID	Requirement	Priority	Notes
FR-TRIP-01	Driver can start trip only when a bus is assigned to their account	P1	"Start Trip" disabled until bus loads
FR-TRIP-02	Trip start activates geolocation watchPosition automatically	P1	Location sharing begins on success
FR-TRIP-03	Location sharing passes bus as parameter, not relying on stale state	P1	Fixes null crash from async timing
FR-TRIP-04	Driver can report delay (1–120 min) with optional reason text	P1	POST /trips/:id/delay
FR-TRIP-05	Delay broadcast to all connected clients via Socket.IO	P1	io.emit — not room-scoped, global
FR-TRIP-06	Ending trip stops geolocation watch and resets all trip state	P1	clearWatch, setCurrentTrip(null)

4.4 Ticket Booking (Passenger)

ID	Requirement	Priority	Notes
FR-BOOK-01	5-step wizard: Route → Bus → Stages → Seats → Confirm & Pay	P1	Back navigation at each step
FR-BOOK-02	Bus list filtered by selected routeId (server-side filter)	P1	?routeId= query param handled in busController
FR-BOOK-03	Stage fare computed via API before seat selection is enabled	P1	Zero-fare or API error blocks Continue button
FR-BOOK-04	Seat grid (47 seats) shows available/booked/selected states	P1	Booked from active trip via /bookings/seats/:tripId
FR-BOOK-05	Multi-seat booking in single transaction with combined total fare	P1	seatNumbers[] array in payload
FR-BOOK-06	E-ticket with real QR code rendered on successful booking	P1	qrcode library, canvas rendering
FR-BOOK-07	E-ticket printable via native window.print() (no popup blocker issue)	P2	Inject/cleanup print CSS dynamically

4.5 Route & Stage Management (Admin)

ID	Requirement	Priority	Notes
FR-ROUTE-01	Routes created via Mapbox geocoding with debounced search	P1	500ms debounce, bbox: Karnataka region
FR-ROUTE-02	Up to 3 route alternatives shown; admin selects preferred path	P2	Mapbox Directions alternatives=true
FR-ROUTE-03	GeoJSON polyline stored in DB and rendered on TrackBus map	P1	Parsed in parsePolyline() utility
FR-ROUTE-04	Route deletion is hard delete (row removed from DB)	P1	getAllRoutes defaults isActive: true

ID	Requirement	Priority	Notes
FR-STAGE-01	Stages pinned by clicking on Leaflet map in pin mode	P1	map.on("click") in pin mode only
FR-STAGE-02	Distance from origin auto-calculated via OSRM on stage creation	P1	HTTPS call to router.project-osrm.org
FR-STAGE-03	Route polyline rendered on stage map for visual reference	P2	L.polyline from stored GeoJSON

4.6 Fleet & Fare Management (Admin)

ID	Requirement	Priority	Notes
FR-BUS-01	Buses created with KA XX F XXXX number format, type, capacity	P1	Split input fields for number parts
FR-BUS-02	Driver and route assignment via searchable dropdowns	P1	Live filter by name/email/number
FR-BUS-03	Bus and route tables paginated (10 per page) with search	P2	Client-side pagination in v1
FR-FARE-01	Price per km configurable per bus type (Ordinary/Express/AC)	P1	Inline card editing with save/cancel
FR-FARE-02	Default fares seeded if collection is empty on first load	P2	Ordinary ₹1.2, Express ₹1.5, AC ₹2.5
FR-FARE-03	Fare summary table shows projections at 40, 130, 250 km	P2	Informational only

5. Non-Functional Requirements

5.1 Performance

ID	Requirement	Target
NFR-P-01	GPS location update frequency	~3–5 seconds (browser watchPosition)
NFR-P-02	ETA recalculation latency	< 5ms (client-side Haversine, no network call)
NFR-P-03	API response time (median)	< 200ms for simple queries, < 500ms for fare calculation
NFR-P-04	Geocoding debounce	500ms after last keystroke
NFR-P-05	Socket.IO ETA broadcast interval	30 seconds (periodic server broadcast)
NFR-P-06	MongoDB query optimization	Indexes on busNumber, email, routeId/stageOrder, ticketId, status/busId

5.2 Security

ID	Requirement	Implementation
NFR-S-01	Admin role blocked at both API and UI layers	HTTP 403 on server; option removed from UI dropdown
NFR-S-02	Auth endpoints rate limited	20 requests per 15 minutes per IP
NFR-S-03	Fare API rate limited + authenticated	30 req/min, JWT required
NFR-S-04	Global API rate limit	200 requests per 15 minutes per IP
NFR-S-05	CORS restricted to configured CLIENT_URL	Comma-separated multi-origin support
NFR-S-06	JWT secret from environment variable	Never hardcoded; minimum 32 characters recommended
NFR-S-07	Seat availability endpoint authenticated	GET /bookings/seats/:tripId requires JWT
NFR-S-08	Passwords hashed with bcrypt	Cost factor 10 (industry standard for web apps)

5.3 Reliability & Availability

ID	Requirement	Implementation
NFR-R-01	Socket.IO client reconnection	5 attempts, 1s delay between attempts (autoConnect: false, reconnection: true)
NFR-R-02	Server graceful shutdown on SIGTERM	Closes HTTP server then MongoDB connection before exit
NFR-R-03	MongoDB startup timeout	5000ms; process.exit(1) on failure
NFR-R-04	Railway restart policy	ON_FAILURE, max 3 retries (railway.toml)
NFR-R-05	Health check endpoint	GET /api/health returns uptime + mongo state

5.4 Usability

- All pages fully functional on screen widths ≥ 320px
- TrackBus sidebar renders as hamburger drawer on mobile (< md breakpoint)
- Skeleton loading animations on all data-fetching pages (SkeletonCard, SkeletonTableRow)
- EmptyState components with actionable CTAs replace plain "No data" strings
- Proper 404 page for all unmatched routes (NotFound component)
- Tab title: "OnTime — Real-Time Bus Tracking" with meta description
- All async actions produce toast notifications (success, error, info)
- Booking zero-fare guard prevents accidental ₹0 tickets
- Delay banner uses z-[9998] fixed positioning to always render above Leaflet map layers

5.5 Maintainability

- Tailwind CSS safelist covers all dynamic gradient and badge color patterns
- Dead trackingStore.js documented and stubbed (was never consumed by any component)
- Environment variables fully documented in .env.example for both client and server
- Route deletion changed from soft-delete to hard-delete for data hygiene consistency
- Reusable UI component library: Skeletons.jsx (SkeletonLine, SkeletonCard, SkeletonTableRow, PageLoader, Spinner, EmptyState)

6. Data Model

6.1 Entity Relationship Overview

> *Relationships: User (1) → (many) Booking | Bus (1) → (many) Trip | Route (1) → (many) Stage | Route (1) → (many) Bus | Trip (1) → (many) Booking | User/Driver (1) → (1) Bus assigned*

6.2 Collections

User

Field	Type	Constraints	Description
_id	ObjectId	Auto	Primary key
name	String	Required, trimmed	Full display name
email	String	Required, unique, lowercase	Login identifier, immutable after creation
password	String	Required, bcrypt hash	Min 6 chars before hashing
phone	String	Optional	Contact number
role	Enum	passenger|driver|admin	Public registration: passenger/driver only
isActive	Boolean	Default: true	Soft-disable without deletion
bookingHistory	ObjectId[]	ref: Booking	Array of booking references

Bus

Field	Type	Constraints	Description
_id	ObjectId	Auto	Primary key
busNumber	String	Required, unique	e.g. KA 09 F 7777
routeId	ObjectId	ref: Route, nullable	Assigned route
driverId	ObjectId	ref: User, nullable	Assigned driver
capacity	Number	Required, min 1	Total passenger seats
currentLocation.coordinates	Object	{lat, lng}	Last known GPS position
currentLocation.timestamp	Date	Nullable	When location was last updated
currentLocation.speed	Number	Default 0	Speed in m/s from GPS
status	Enum	active|inactive|maintenance	Operational status
isOnTrip	Boolean	Default false	Whether bus has an active trip
currentTripId	ObjectId	ref: Trip, nullable	Active trip reference
busType	Enum	Ordinary|Express|AC	Determines fare per km rate

Route

Field	Type	Constraints	Description
routeName	String	Required	e.g. Mysuru → Mangaluru

Field	Type	Constraints	Description
routeNumber	String	Required, unique	e.g. RT-101
sourceCity	String	Optional	Human-readable source city name
destinationCity	String	Optional	Human-readable destination name
sourceCoordinates	Object	{lat, lng}	GPS coords of source point
destinationCoordinates	Object	{lat, lng}	GPS coords of destination
polyline	String	GeoJSON string	Encoded route path from Mapbox Directions
stops	Array	[{name, coordinates, order}]	Legacy fallback stop list
distance	Number	km, default 0	Total route distance
estimatedDuration	Number	minutes, default 0	Expected travel time
isActive	Boolean	Default true	getAllRoutes filters isActive: true by default

Stage

Field	Type	Constraints	Description
routeId	ObjectId	ref: Route, required	Parent route
stageName	String	Required	e.g. Hunsur Bus Stand
latitude	Number	Required, -90 to 90	GPS latitude of stop
longitude	Number	Required, -180 to 180	GPS longitude of stop
stageOrder	Number	Required, min 0	Sequence position on route (0 = first)
distanceFromOrigin	Number	km, auto-calculated via OSRM	Used for fare distance calculation

Trip

Field	Type	Constraints	Description
busId	ObjectId	ref: Bus, required	Bus on this trip
routeId	ObjectId	ref: Route, required	Route being followed
driverId	ObjectId	ref: User, required	Driving this trip
startTime	Date	Default: now	Trip start timestamp
endTime	Date	Default null	Populated when trip ends
status	Enum	scheduled|in-progress|completed|cancelled	Lifecycle state
bookedSeats	Number[]	Seat numbers 1–47	Seats reserved on this trip
locationHistory	Array	[{lat,lng,timestamp,speed}]	Full GPS trail
delayMinutes	Number	Default 0	Current reported delay
delayReason	String	Default empty	Optional delay explanation

Booking

Field	Type	Constraints	Description
userId	ObjectId	ref: User, required	Booking passenger
busId	ObjectId	ref: Bus, required	Bus booked on
routeId	ObjectId	ref: Route, required	Route of travel
tripId	ObjectId	ref: Trip, nullable	Active trip at time of booking (optional)
seatNumber	Number	Required (legacy)	First seat number (backward compat)
seatNumbers	Number[]	Default []	All booked seat numbers
fromStop	String	Default empty	Boarding stage name
toStop	String	Default empty	Destination stage name
travelDate	Date	Required	Date of travel
status	Enum	confirmed|cancelled|completed	Booking lifecycle
paymentStatus	Enum	pending|paid|refunded	Payment state
amount	Number	Required, min 0	Total fare paid (₹)
ticketId	String	Unique, required	TKT + timestamp + 4-digit random
passengerDetails	Object	{name, phone}	Stored for e-ticket display

BusTypeFare

Field	Type	Constraints	Description
busType	String	Required, unique	Type name: Ordinary / Express / AC / custom
pricePerKM	Number	Required, min 0	Fare in ₹ per kilometer

7. API Reference

Base URL: /api | Auth: JWT Bearer token in Authorization header

7.1 Auth — /api/auth

Method	Path	Auth	Description
POST	/register	Public	Register passenger or driver. Admin role returns 403.
POST	/login	Public	Returns JWT token + user object.
GET	/profile	JWT	Own profile with booking history.
PUT	/profile	JWT	Update name and phone.
PUT	/profile/password	JWT	Change password (requires currentPassword).
GET	/users?role=	JWT	List users by role (used by admin for driver assignment).

7.2 Routes — /api/routes

Method	Path	Auth	Description
GET	/	Public	All routes where isActive: true. Pass ?isActive=false to include deleted.
GET	/:id	Public	Single route by ID.
POST	/	Admin	Create route with Mapbox polyline.
PUT	/:id	Admin	Update route name, number, or geometry.
DELETE	/:id	Admin	Hard delete route (removes from DB).

7.3 Buses — /api/buses

Method	Path	Auth	Description
GET	/?status=&routeId=&isOnTrip=	Public	All buses. Supports status, routeId, isOnTrip filters.
GET	/:id	Public	Single bus with populated route and driver.
POST	/	Admin	Create bus (validates driverId is a driver role).
PUT	/:id	Admin	Update bus route, capacity, status, type.
PUT	/:id/driver	Admin	Assign driver to bus.
PUT	/:id/location	Driver	REST fallback for location update (Socket.IO preferred).

7.4 Trips — /api/trips

Method	Path	Auth	Description
GET	/active	Public	All in-progress trips with populated bus, route, driver.
POST	/start	Driver	Start trip for own bus. Fails if bus already on trip.
PUT	/:id/end	Driver	End own trip. Sets bus.isOnTrip = false.
GET	/my-current	Driver	Own currently active trip.
POST	/:id/delay	Driver	Report delay. Validates ownership. Broadcasts trip:delay.
GET	/:id/delay	JWT	Get current delay for a trip.
GET	/history	Admin	Trip history with filters (status, busId, driverId).

7.5 Bookings — /api/bookings

Method	Path	Auth	Description
POST	/	JWT	Create booking. Accepts seatNumbers[]. Optional tripId for seat locking.
GET	/my	JWT	Own bookings, sorted newest first. Supports ?status= filter.
GET	/seats/:tripId	JWT	Booked seat numbers for a trip (for seat map UI).
GET	/:id	JWT	Single booking (own only, or admin).
PUT	/:id/cancel	JWT	Cancel own booking. Releases seats from trip.

7.6 Stages — /api/stages

Method	Path	Auth	Description
GET	/:routeId	Public	All stages for a route, ordered by stageOrder.
POST	/	Admin	Create stage. OSRM calculates distanceFromOrigin automatically.
PUT	/:id	Admin	Update stage name, coordinates, or order.
DELETE	/:id	Admin	Hard delete stage.

7.7 Fare — /api/fare

Method	Path	Auth	Description
GET	/calculate?fromStageId=&toStageId=&busId=	JWT	Fare = (toStage.distanceFromOrigin - fromStage.distanceFromOrigin) × pricePerKM. Rate limited: 30/min.

7.8 Bus Type Fares — /api/bus-types

Method	Path	Auth	Description
GET	/	Public	All configured bus type fare rates.
POST	/	Admin	Create fare rate. Fails if busType already exists.
PUT	/:id	Admin	Update pricePerKM for a bus type.

8. Socket.IO Event Catalog

8.1 Connection Lifecycle

Socket connects automatically for authenticated users via useNotifications hook on Navbar mount. For unauthenticated users, the socket does NOT connect (useNotifications guards on isAuthenticated). On logout, socket.disconnect() is called.

8.2 Client → Server Events

Event	Payload	Sender	Description
driver:location-update	{ busId, lat, lng, speed }	Driver	GPS position update from watchPosition. Updates Bus.currentLocation in DB.
driver:trip-start	{ tripId, busId }	Driver	Emitted after successful POST /trips/start. Broadcasts trip:started.
driver:trip-end	{ tripId, busId }	Driver	Emitted after successful PUT /trips/:id/end. Broadcasts trip:ended.
driver:report-delay	{ tripId, busId, busNumber, routeName, delayMinutes, delayReason }	Driver	Alternative socket-based delay report (REST endpoint is preferred).
track:bus	{ busId }	Passenger	Joins socket room bus-{busId}. Used for future scoped broadcasts.
untrack:bus	{ busId }	Passenger	Leaves socket room bus-{busId}.

8.3 Server → Client Events (Broadcast)

Event	Payload	Trigger	Consumer
bus:location-updated	{ busId, busNumber, location: {lat, lng, speed, timestamp} }	driver:location-update received	TrackBus — updates marker position + triggers ETA recalc
trip:started	{ tripId, busId, timestamp }	driver:trip-start received	Optional consumer — not currently handled in UI
trip:ended	{ tripId, busId, timestamp }	driver:trip-end received	Optional consumer — not currently handled in UI
trip:delay	{ tripId, busId, busNumber, routeName, delayMinutes, delayReason, timestamp }	POST /trips/:id/delay OR driver:report-delay	TrackBus — DelayBanner; useNotifications — bell + browser push
bus:eta-updated	{ busId, tripId, currentLocation, speed }	Server interval every 30s	Not currently consumed in UI (client calculates own ETA)

9. Deployment & Infrastructure

9.1 Frontend — Vercel

Setting	Value
Build Command	vite build
Output Directory	dist/
SPA Routing	vercel.json rewrite: /* → /index.html
Node Requirement	^20.19.0 or >=22.12.0 (Vite 7 requirement)
Environment Variables	VITE_API_URL, VITE_SOCKET_URL, VITE_MAPBOX_TOKEN (set in Vercel dashboard)

9.2 Backend — Railway

Setting	Value
Builder	NIXPACKS
Start Command	node server.js
Health Check Path	/api/health
Health Check Timeout	300 seconds
Restart Policy	ON_FAILURE, max 3 retries
Node Requirement	>= 18
Environment Variables	MONGODB_URI, JWT_SECRET, PORT, NODE_ENV, CLIENT_URL (comma-separated for multi-origin)

9.3 Database — MongoDB Atlas

Setting	Value
Tier	M0 (free) for development; M10+ recommended for production
Connection	TLS required (tlsAllowInvalidCertificates: true in dev)
Timeout	5000ms serverSelectionTimeout
Indexes	busNumber, email, routeId+stageOrder, status+busId, ticketId
Region	Same AWS region as Railway deployment for optimal latency

9.4 Environment Variables Reference

Client (client/.env)

```
VITE_API_URL=http://localhost:5000/api
VITE_SOCKET_URL=http://localhost:5000
VITE_MAPBOX_TOKEN=pk.eyJ1...
```

Server (server/.env)

```
MONGODB_URI=mongodb+srv://<user>:<pass>@<cluster>/ontime?retryWrites=true&w=maj
ority
JWT_SECRET=<minimum-32-character-random-string>
PORT=5000
NODE_ENV=development
CLIENT_URL=http://localhost:5173
```

10. Known Limitations & Future Roadmap

10.1 Known Limitations (v1.0)

#	Limitation	Impact	Suggested Fix
L-01	Payment gateway simulated	No real money handling; bookings go to confirmed without payment	Integrate Razorpay or Stripe checkout
L-02	Seat booking allowed without active trip	Seats not physically locked to trip; double-booking theoretically possible if two passengers book simultaneously	Add optimistic lock on Trip.bookedSeats using findOneAndUpdate with $addToSet
L-03	ETA uses straight-line distance	ETA inaccurate on winding roads; can under-estimate by 20–40%	Route-snapping with OSRM or Mapbox Directions for road-network ETA
L-04	Browser push notifications require open tab	Passengers cannot receive delay alerts when browser is closed	Integrate Web Push (service worker) with VAPID keys
L-05	Socket.IO single server instance	Cannot horizontally scale without session state sharing	Add socket.io-redis-adapter for multi-instance support
L-06	Route deletion does not cascade to buses	Buses may remain assigned to deleted routes	Cascade update Bus.routeId = null when route is deleted
L-07	No real-time seat occupancy push	Seat map must be refreshed manually after another passenger books	Emit seat-booked event from booking creation handler
L-08	OSRM public instance dependency	rate-limited and unreliable for production scale	Self-host OSRM on a VPS with Karnataka OSM data extract
L-09	Mapbox token exposed in client bundle	Token visible in browser; could be abused	Add Mapbox token restriction to specific URLs/domains in Mapbox dashboard
L-10	No email/SMS OTP verification on register	Anyone can register with any email	Add email verification flow with Resend or Nodemailer

10.2 Future Roadmap

Priority	Feature	Description
P1	Real Payment Gateway	Integrate Razorpay for UPI, card, and netbanking payments
P1	Email Verification	Send OTP on registration to verify email ownership
P1	Service Worker Push Notifications	Background delay alerts using Web Push API even when browser is closed
P2	Road-Network ETA	OSRM or Mapbox Directions-based real-time ETA snapped to road
P2	Real-time Seat Push	Emit socket event when seats are booked to update all open booking wizards live
P2	Admin User Management	View, activate, deactivate, and role-change users from admin panel
P2	Driver Mobile App	PWA or React Native app optimised for driver GPS sharing on mobile
P3	Multi-city Expansion	Expand geocoding bbox beyond Karnataka; add city/state filters

Priority	Feature	Description
P3	Analytics Dashboard	Trip metrics, revenue by route, popular stages, peak times
P3	Seat Class Pricing	Window/aisle seat surcharge; front-row premium pricing
P3	SMS Booking Confirmation	Send ticket details via SMS after successful booking using Twilio/MSG91
P3	Cancellation Refunds	Automated refund calculation based on hours before travel date

Appendix A: Fare Calculation Formula

A.1 Stage-to-Stage Fare

The fare between two stages is calculated as:

> *fare = ROUND((toStage.distanceFromOrigin - fromStage.distanceFromOrigin) × pricePerKM)*

Where distanceFromOrigin is calculated via OSRM road-network routing from the route source coordinates at the time each stage is created. This distance is stored on the Stage document and is not recalculated dynamically.

A.2 Default Fare Rates

Bus Type	Price/km	40 km	130 km	250 km
Ordinary	₹1.20	₹48	₹156	₹300
Express	₹1.50	₹60	₹195	₹375
AC	₹2.50	₹100	₹325	₹625

Appendix B: ETA Calculation

B.1 Haversine Formula

ETA is computed entirely on the client in etaCalculator.js:

```
distance_km = haversine(currentLat, currentLng, destLat, destLng)
speedKmh = speed_ms × 3.6   // GPS speed in m/s converted to km/h
eta_minutes = ROUND( (distance_km / speedKmh) × 60 )
```

If speed < 1 km/h, the bus is considered stationary and ETA is displayed as "Stationary". If no destination coordinates exist on the route, ETA returns null.

Appendix C: Seat Numbering Layout

The 47-seat bus grid layout used in BookTicket (seat numbers 1–47):

Row layout (top to bottom, left = driver side):

```
Row 1:  1,  8,  9, 16, 17, 24, 25, 32, 33, 40, [aisle], 47
Row 2:  2,  7, 10, 15, 18, 23, 26, 31, 34, 39, [aisle], 46
Row 3:  [driver seat row / skip seats]           [aisle], 45
Row 4:  3,  6, 11, 14, 19, 22, 27, 30, 35, 38, 41, 44
Row 5:  4,  5, 12, 13, 20, 21, 28, 29, 36, 37, 42, 43
```